Things That Look Similar in Backend Engineering But Aren't

The distinctions that matter when you're building real systems.

01 — Authentication vs Authorization

They both control access. But they answer different questions.

Authentication

→ Who are you?

Authorization

→ What are you allowed to do?

Example:

Login with password

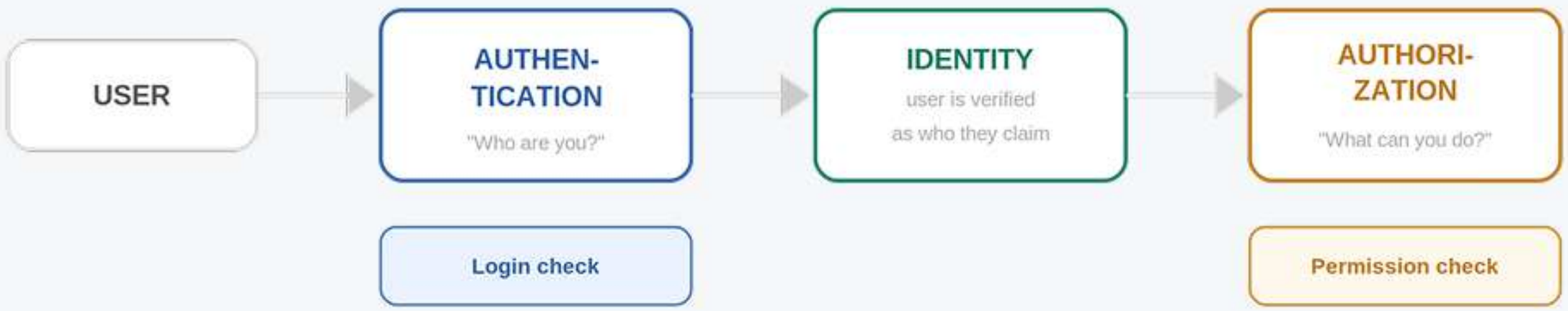
→ Authentication

Can this user delete this post?

→ Authorization

The difference matters because:

A user can be authenticated and still not be authorized to perform an action.



@this.girl.tech

02 — Latency vs Throughput

They both describe performance. But they measure different things.

Latency

→ How long does one operation take?

Throughput

→ How much work can the system handle over time?

Example:

An API responds in 100 ms

→ latency

An API handles 10,000 requests/sec

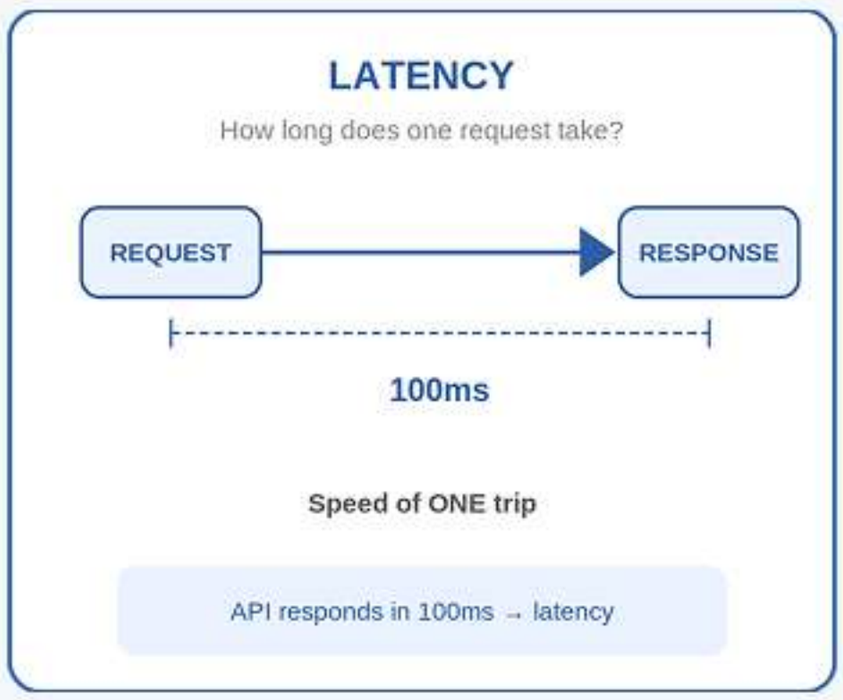
→ throughput

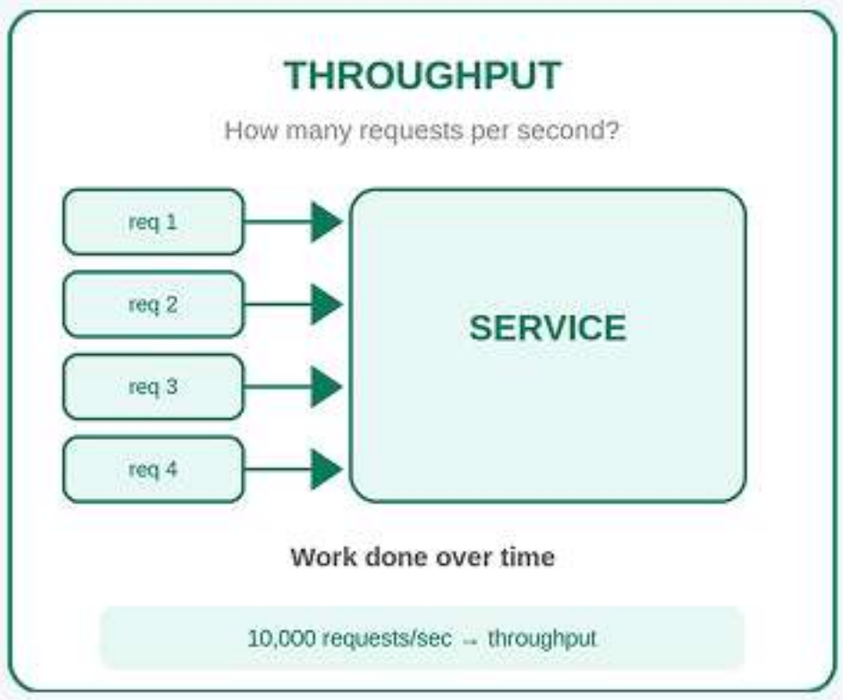
Think of it as:

Latency = speed of one trip

Throughput = how many trips you can handle

A system can have low latency but limited throughput, or high throughput while individual requests are slower.





@this.girl.tech

03 — Retry vs Redelivery

They both look like "try it again." But the source of the second attempt is different.

Retry

→ The client/producer intentionally attempts the operation again.

Redelivery

→ A messaging system delivers a message again because it wasn't successfully acknowledged/processed.

Example:

API request fails → client sends again

= Retry

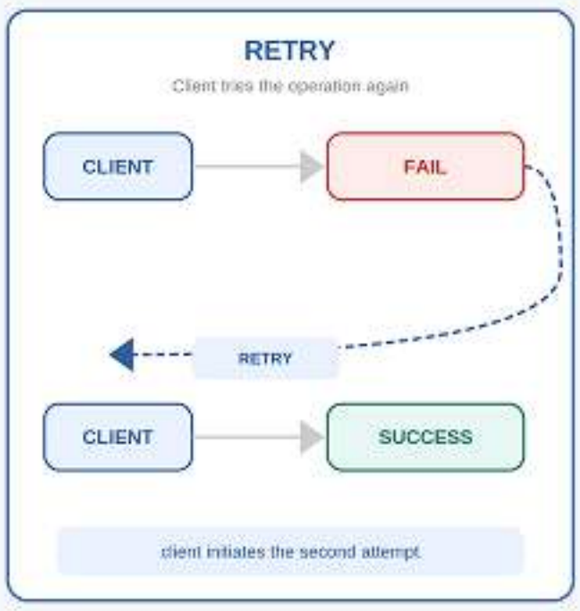
Message isn't acknowledged → queue sends it again

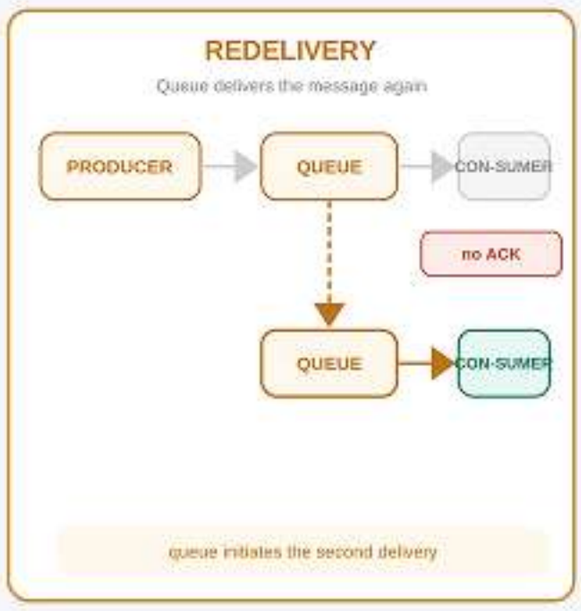
= Redelivery

Why it matters:

A redelivered message may be the same message being processed again.

That means your consumer may need to handle duplicate processing safely.





@this.girl.tech

04 — Timeout vs Cancellation

Both stop waiting for work. But they don't mean the same thing.

Timeout

→ The operation exceeded a time limit.

Cancellation

→ Someone/system requests that the operation stop.

Example:

Request hasn't finished after 5s

→ Timeout

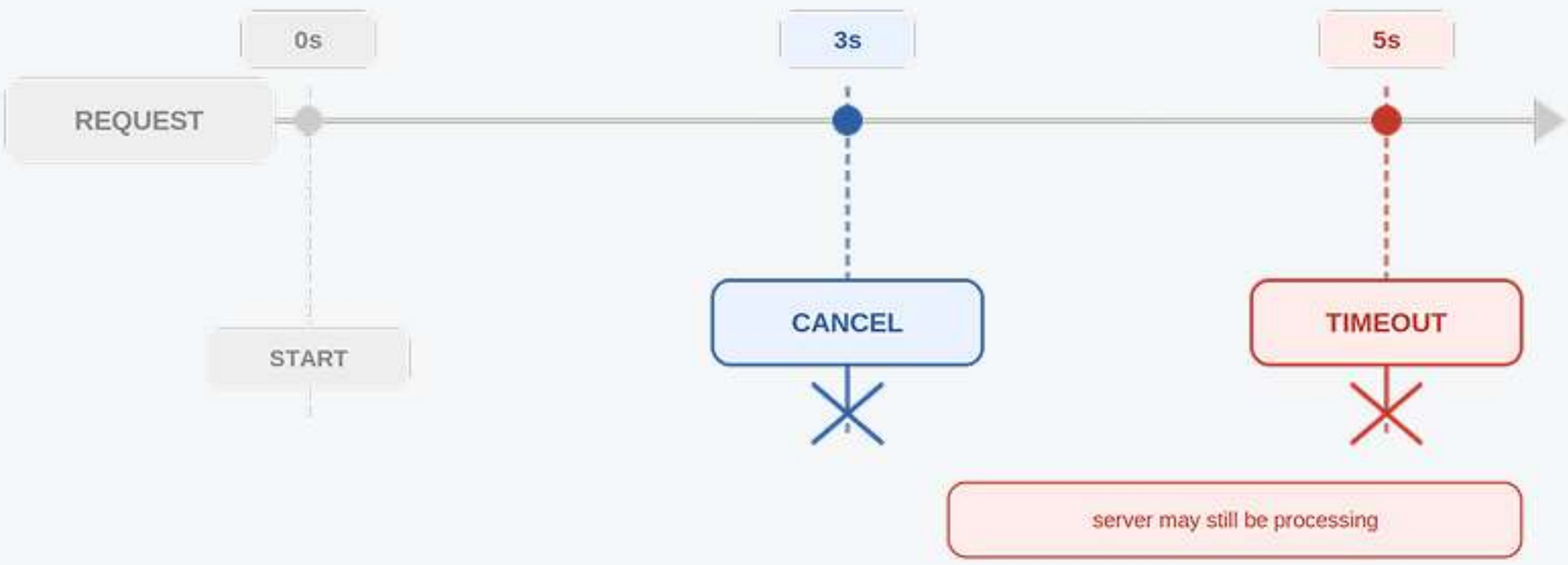
User closes the page

→ Request may be cancelled

Important:

A timeout doesn't automatically mean the underlying work stopped.

The caller may have stopped waiting while the server is still processing.



@this.girl.tech

05 — Rate Limiting vs Backpressure

Both control traffic. But they solve different problems.

Rate Limiting

→ Controls how much traffic is allowed in.

Backpressure

→ Signals that a downstream component can't keep up.

Example:

100 requests/sec allowed per user

→ Rate limiting

Consumer is overloaded → producer slows down

→ Backpressure

Think:

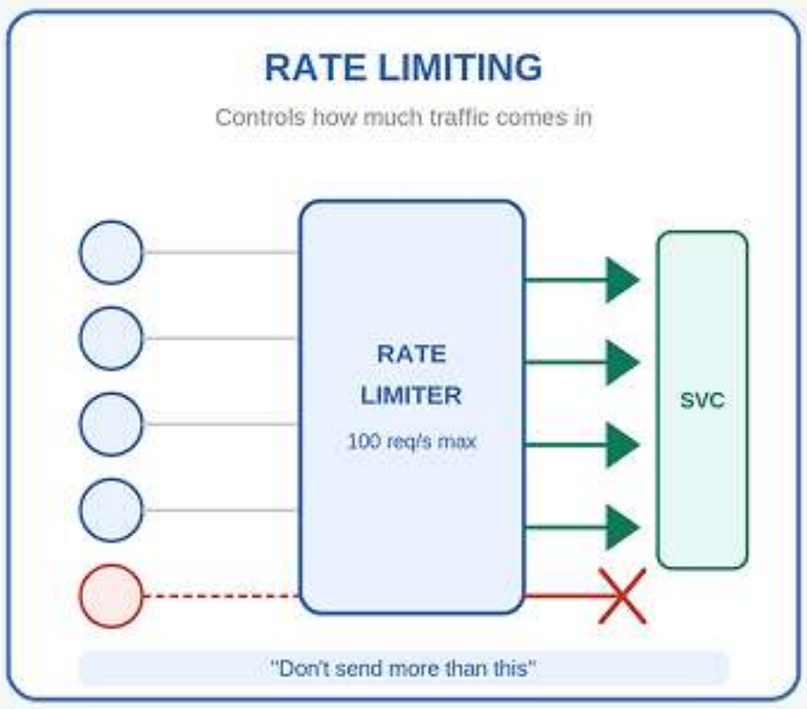
Rate limiting:

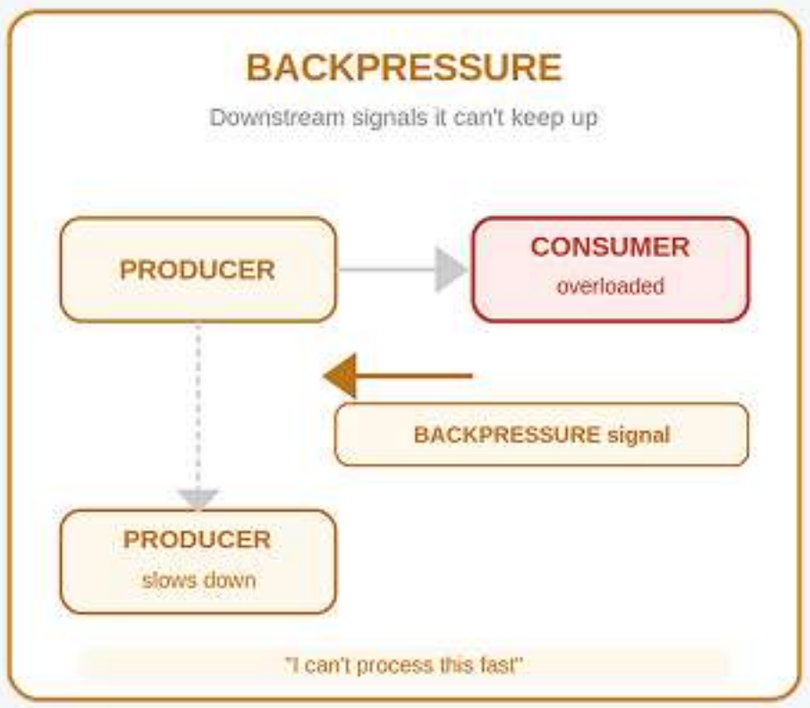
“Don't send more than this.”

Backpressure:

“I can't process this fast. Slow down.”

They can exist in the same system.





@this.girl.tech

06 — Queue vs Stream

Both carry messages/events. But their models are different.

Queue

→ Work is generally consumed for processing.

Stream

→ Events are retained as a sequence that consumers can read independently and, depending on the system, re-read.

Example:

Queue

Job → Worker → processed

Another worker generally doesn't process that same job after successful consumption.

Stream

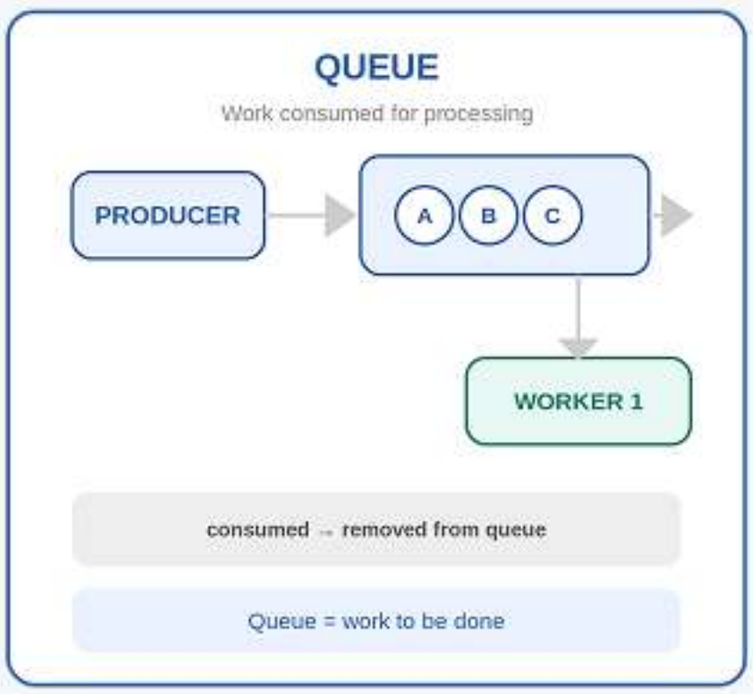
Event A → Event B → Event C

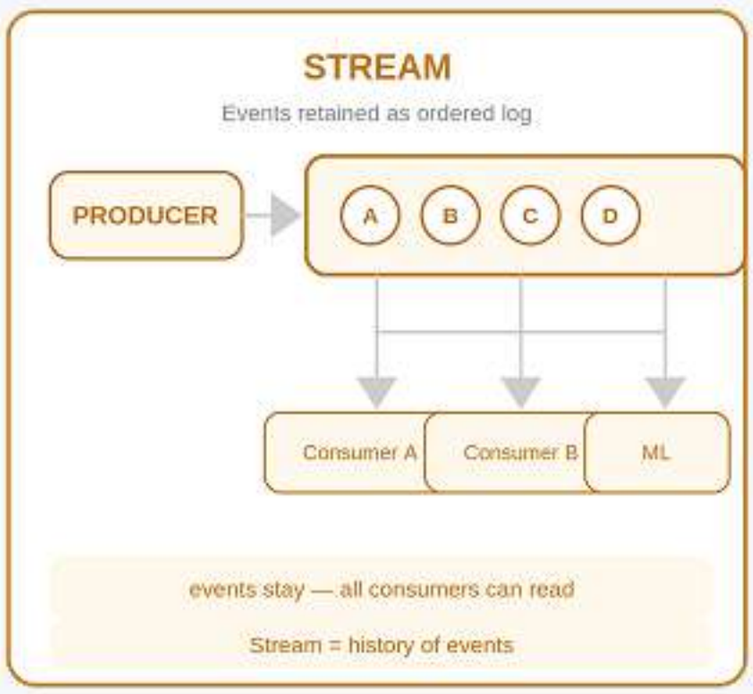
Multiple consumers can independently read the event history.

Mental model:

Queue = work waiting to be done

Stream = history of events





@this.girl.tech

07 — Replication vs Sharding

Both distribute data across machines. But they distribute it differently.

Replication

→ Multiple machines keep copies of the same data.

Sharding

→ Different machines keep different portions of the data.

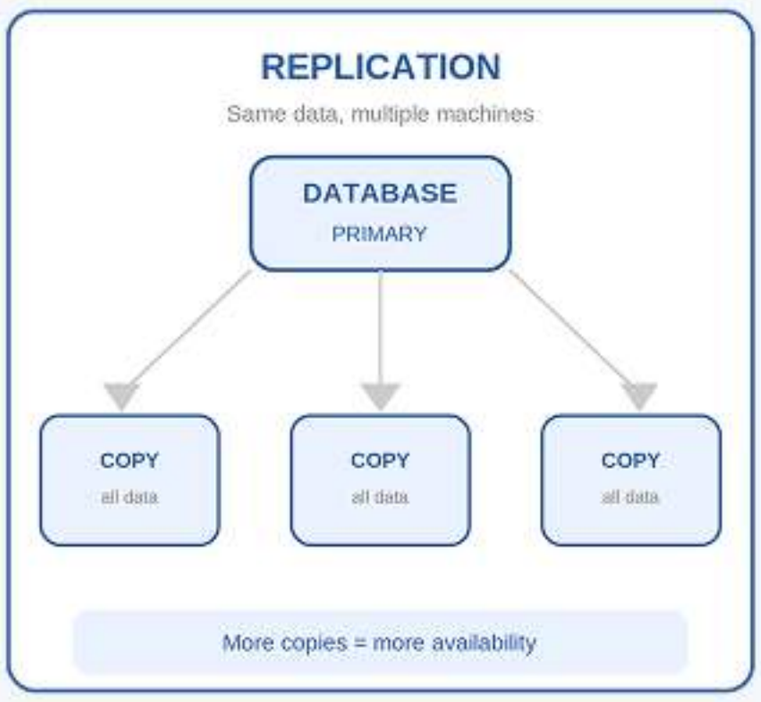
Example:

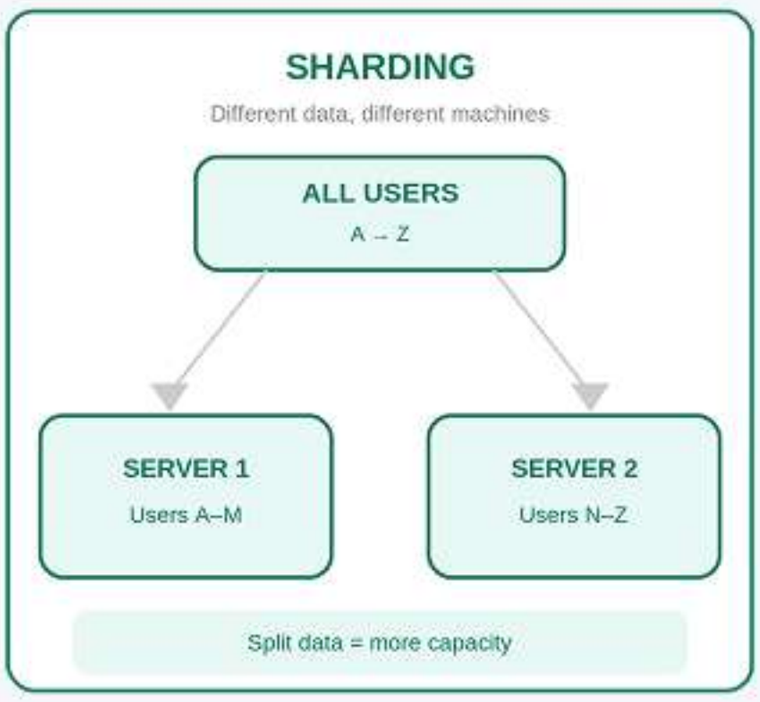
Replication

Database A

↓ ↓ ↓

Copy Copy Copy





Sharding

Users A–M → Server 1

Users N–Z → Server 2

Think:

Replication = more copies

Sharding = split the data

They can also be used together.

@this.girl.tech

08 — Event vs Command

Both can be messages. But their meaning is completely different.

Command

→ “Please do this.”

Event

→ “This already happened.”

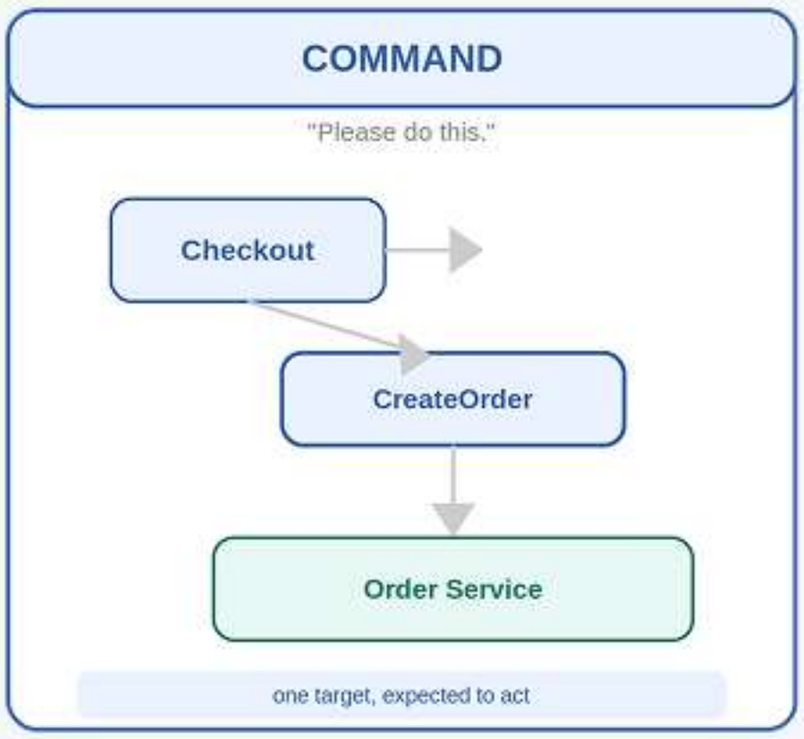
Example:

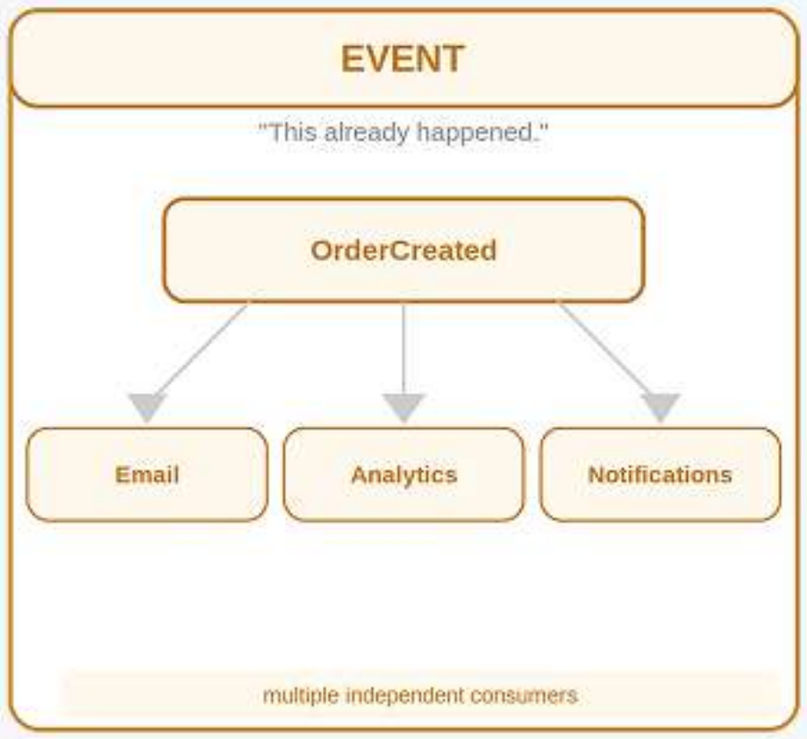
Command:

CreateOrder

Event:

OrderCreated





A command expresses intent.

An event communicates a fact.

Why this matters:

A command has a target that is expected to act.

An event can have multiple independent consumers.

@this.girl.tech

Similar words. Different problems. Understanding the difference helps you make better engineering decisions.

Learn the distinction.

Then choose the right tool.

📌 ***Save this for your next backend project.***

💬 ***Comment “MORE” if you want another part.***

@this.girl.tech